

High Level Design (HLD)

British Auction RFQ System

Table of Contents

1. System Overview
2. Architecture Overview
3. Core Components
 - 3.1 API Layer
 - 3.2 Service Layer
 - 3.2.1 AuctionEngineService
 - 3.2.2 RankingService
 - 3.2.3 ExtensionService
 - 3.2.4 ValidationService
 - 3.3 Data Models
4. Data Flow — Bid Placement
5. Concurrency & Consistency
 - 5.1 Challenges
 - 5.2 Solutions
6. Key Algorithms
 - 6.1 Ranking Algorithm
 - 6.2 Extension Logic
 - 6.3 Complexity & Design Benefit
7. Real-Time Strategy
 - 7.1 Current Approach
 - 7.2 Limitations
 - 7.3 Future Enhancement
8. Security
 - 8.1 Authentication
 - 8.2 Authorization
 - 8.3 Data Protection
9. Tradeoffs
10. Scalability Strategy
11. Failure Handling
12. Conclusion

1. System Overview

The British Auction RFQ (Request for Quotation) System is a real-time bidding platform designed to ensure **fair competition, strong consistency, and deterministic closure** in supplier auctions.

Traditional auction systems suffer from *bid sniping*, where participants place bids at the last second, preventing others from responding. This leads to unfair outcomes and reduced competition.

To address this, the system introduces:

- A **dynamic extension mechanism** that increases auction time when competitive bids are placed near closing
- A **forced closure constraint** to prevent indefinite extensions

This ensures that every participant has a fair opportunity to respond while maintaining predictable auction timelines.

The system is designed as a **state-driven system**, where each bid triggers controlled transitions in auction state, ensuring correctness even under concurrent access.

2. Architecture Overview

The system follows a **layered architecture** to enforce separation of concerns and maintain scalability.

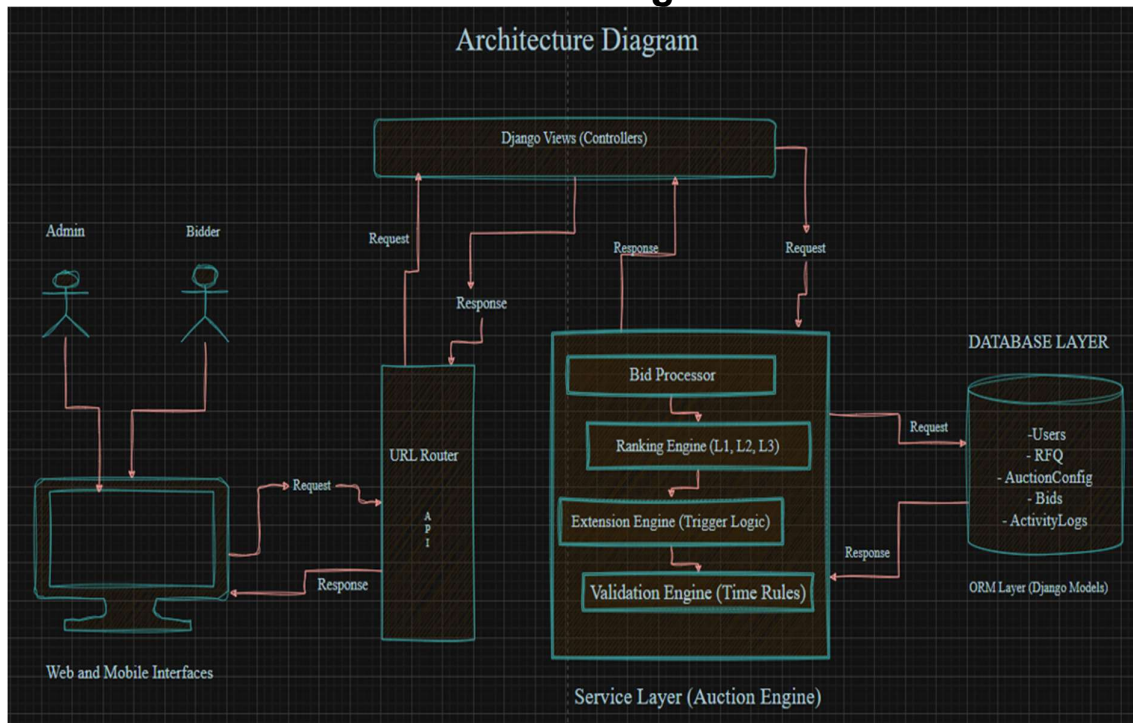
Layers:

- **Client Layer**
 - Browser-based UI with countdown timers and polling
 - Displays rankings and auction status
- **API Layer**
 - Built using Django REST Framework
 - Handles request routing, authentication, and response formatting
 - Keeps business logic minimal
- **Service Layer (Core Intelligence)**
 - Contains all business logic
 - Responsible for bid processing, ranking, and extension handling
 - Ensures deterministic behavior
- **Database Layer**
 - Django ORM models representing Auctions, Bids, Events
 - Maintains relationships and constraints

Key Design Decision:

All domain logic is centralized in the **Service Layer**, avoiding tightly coupled and hard-to-maintain code in views or models.

Architecture Diagram



3. Core Components

The system is composed of the following key components:

3.1 API Layer

- Handles authentication and permissions
- Processes incoming requests
- Delegates all business logic to services

3.2 Service Layer

AuctionEngineService

- Acts as the central orchestrator
- Manages the entire bid lifecycle
- Ensures atomic execution of operations

3.3 RankingService

- Computes auction rankings efficiently
- Considers only the latest bid from each bidder
- Optimized for performance

3.4 ExtensionService

- Implements auction fairness logic
- Extends auction time based on trigger rules:
 - Bid Received
 - Rank Change
 - L1 Change

3.5 ValidationService

- Validates auction state and bid inputs
- Prevents invalid or inconsistent operations

Data Models

- **Auction**: Stores auction metadata and timing
- **Bid**: Stores supplier quotes
- **AuctionEvent**: Logs all system activities
- **AuctionConfig**: Stores extension rules

4. Data Flow — Bid Placement

The bid placement flow is the most critical operation in the system.

Flow Steps:

1. Bidder submits a bid via API
2. API layer validates input and forward's request
3. AuctionEngineService processes the bid inside an atomic transaction
4. Auction row is locked to prevent concurrent conflicts
5. Existing auction state (rankings, L1) is captured
6. New bid is stored
7. Rankings are recomputed
8. System detects changes (rank change or L1 change)
9. Extension logic is evaluated and applied if needed
10. Events are logged for audit purposes
11. Transaction is committed

Key Insight:

The entire flow is treated as a **single atomic unit**, ensuring that partial updates do not occur.

5. Concurrency & Consistency

Handling concurrency is critical due to simultaneous bid submissions.

5.1 Challenges:

- Multiple users placing bids at the same time
- Maintaining correct rankings
- Avoiding duplicate or inconsistent states

5.2 Solutions:

- **Atomic Transactions**
 - All operations executed within a single transaction
 - Ensures all-or-nothing execution
- **Row-Level Locking**
 - Auction row locked using `select_for_update()`
 - Prevents simultaneous modifications
- **Deterministic State Updates**
 - Rankings and extensions computed within the transaction
 - No reliance on external or asynchronous updates

Outcome:

System guarantees **strong consistency**, even under high concurrency.

6. Key Algorithms

6.1 Ranking Algorithm

Instead of processing all bids, the system:

1. Fetches the latest bid per bidder
2. Sorts based on total cost and timestamp

6.2 Complexity:

- Efficient: $O(n \log n)$
- Avoids redundant computation

6.2 Extension Logic

Three configurable trigger types:

- **Bid Received** → extend on any bid
- **Rank Change** → extend when rankings change
- **L1 Change** → extend only when top bidder changes

6.3 Design Benefit:

Provides flexibility to tune fairness vs strictness.

7. Real-Time Strategy

7.1 Current Approach:

- Polling every few seconds
- Updates countdown timer and rankings

7.2 Limitations:

- Increased server load
- Delayed updates

7.3 Future Enhancement:

- WebSockets for real-time push updates
- Event-driven architecture

8. Security

Authentication

- Django authentication system
- Session-based security

Authorization

- Role-based access:

- Auctioneer
- Bidder

Data Protection

- ORM prevents SQL injection
- Input validation ensures correctness
- Decimal-based pricing ensures accuracy

10. Scalability Strategy

Future improvements include:

- Database → PostgreSQL
- Caching → Redis
- Async processing → Celery
- Real-time → WebSockets
- Load balancing → Horizontal scaling

Design Advantage:

Layered architecture allows scaling without rewriting core logic.